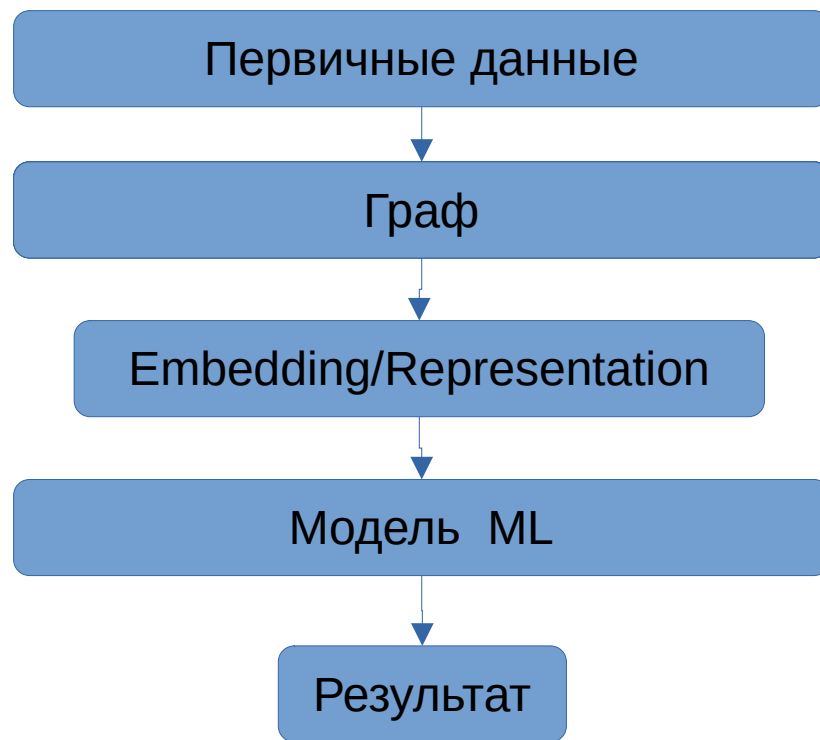
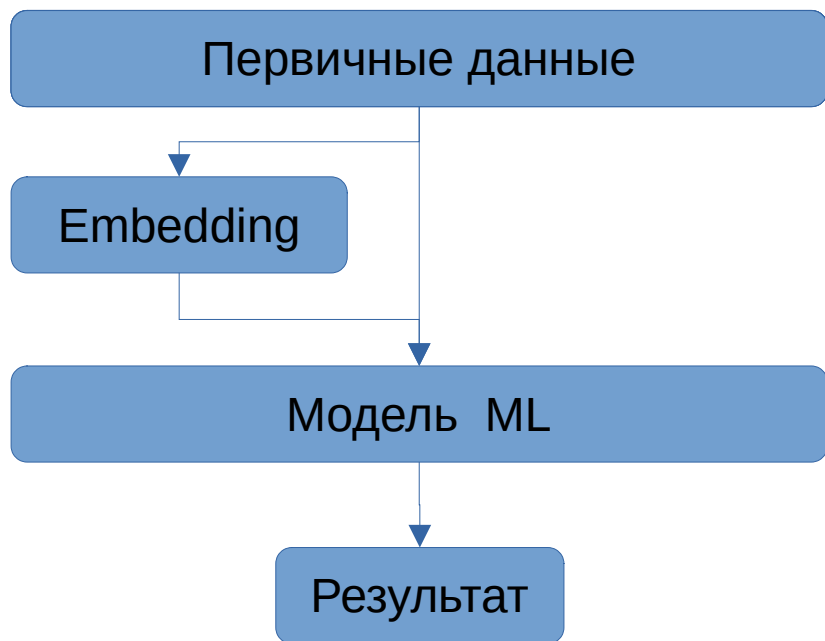
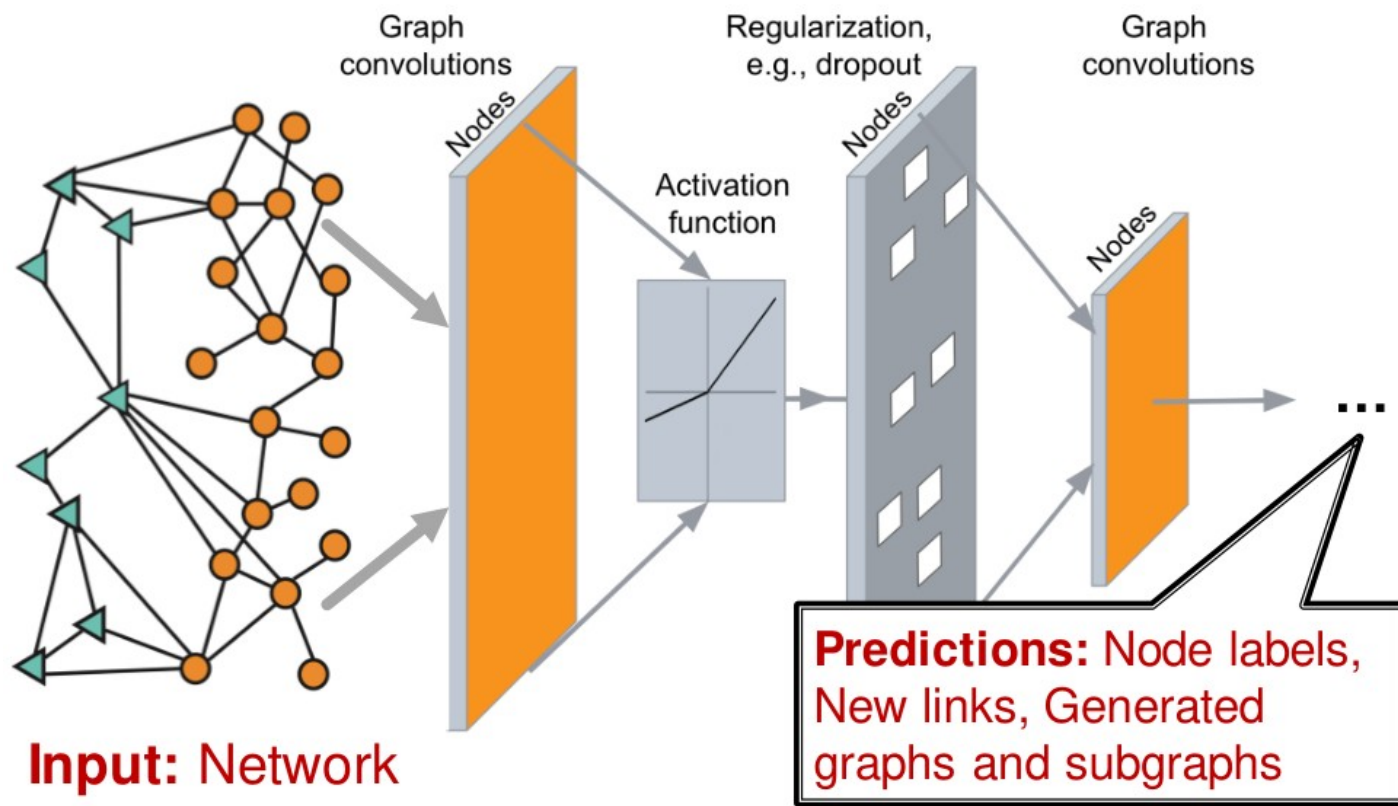


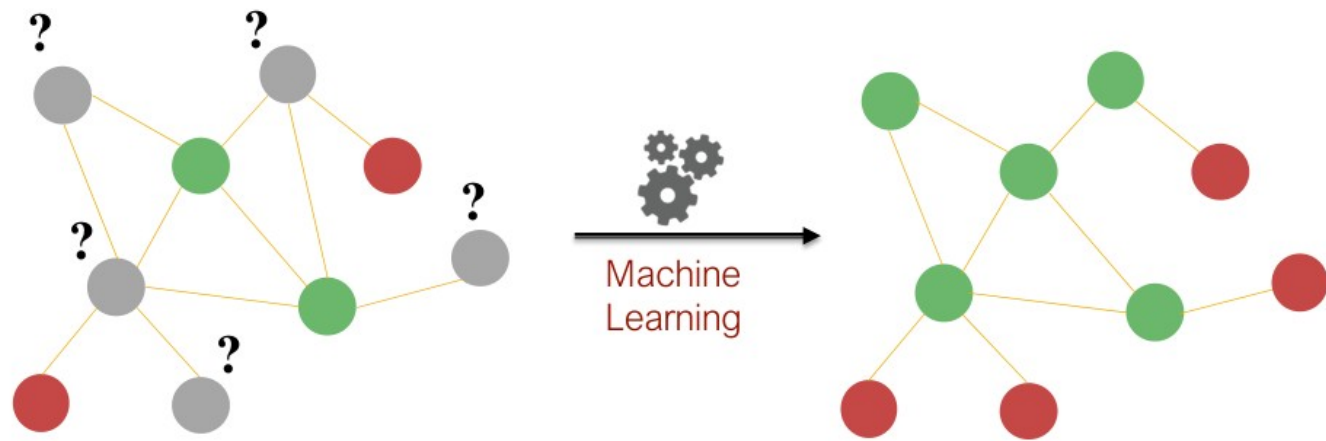
Машинное обучение на графах



- **Node classification**: Predict a property of a node
 - **Example**: Categorize online users / items
- **Link prediction**: Predict whether there are missing links between two nodes
 - **Example**: Knowledge graph completion
- **Graph classification**: Categorize different graphs
 - **Example**: Molecule property prediction
- **Clustering**: Detect if nodes form a community
 - **Example**: Social circle detection
- **Other tasks**:
 - **Graph generation**: Drug discovery
 - **Graph evolution**: Physical simulation



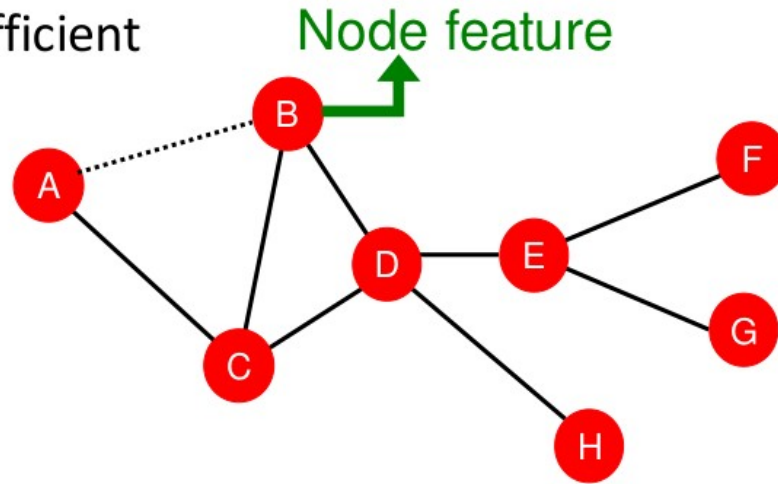
Классические признаки



Node classification

Классические признаки для ноды

- Node degree
- Node centrality
- Clustering coefficient
- Graphlets



- Importance-based features:

- Node degree
- Different node centrality measures

- Structure-based features:

- Node degree
- Clustering coefficient
- Graphlet count vector

Useful for predicting influential nodes in a graph

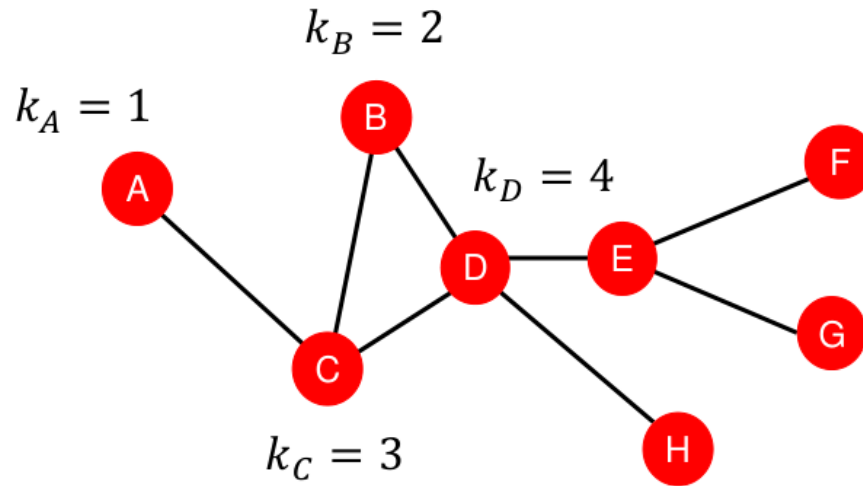
- **Example:** predicting celebrity users in a social network

Useful for predicting a particular role a node plays in a graph:

- **Example:** Predicting protein functionality in a protein-protein interaction network.

Степень вершины

- The degree k_v of node v is the number of edges (neighboring nodes) the node has.
- Treats all neighboring nodes equally.



Центральность вершины

Eigenvector centrality:

- A node v is important if **surrounded by important neighboring nodes** $u \in N(v)$.
- We model the centrality of node v as **the sum of the centrality of neighboring nodes**:

$$c_v = \frac{1}{\lambda} \sum_{u \in N(v)} c_u$$

λ is normalization constant (it will turn out to be the largest eigenvalue of A)

$$c_v = \frac{1}{\lambda} \sum_{u \in N(v)} c_u$$

λ is normalization const
(largest eigenvalue of A)



$$\lambda \mathbf{c} = \mathbf{A} \mathbf{c}$$

- A : Adjacency matrix
 $A_{uv} = 1$ if $u \in N(v)$
- $\mathbf{c}$: Centrality vector
- λ : Eigenvalue

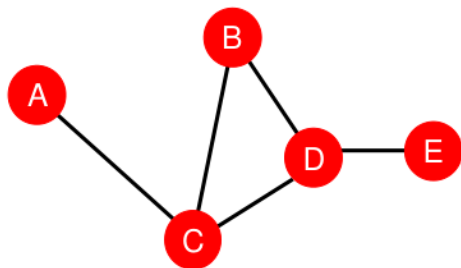
- We see that centrality $\mathbf{c}$ is the **eigenvector of A !**
- The largest eigenvalue λ_{max} is always positive and unique (by Perron-Frobenius Theorem).
- The eigenvector $\mathbf{c}_{max}$ corresponding to λ_{max} is used for centrality.

■ Betweenness centrality:

- A node is important if it lies on many shortest paths between other nodes.

$$c_v = \sum_{s \neq v \neq t} \frac{\#(\text{shortest paths between } s \text{ and } t \text{ that contain } v)}{\#(\text{shortest paths between } s \text{ and } t)}$$

- **Example:**



$$c_A = c_B = c_E = 0$$

$$c_C = 3$$

(A-C-B, A-C-D, A-C-D-E)

$$c_D = 3$$

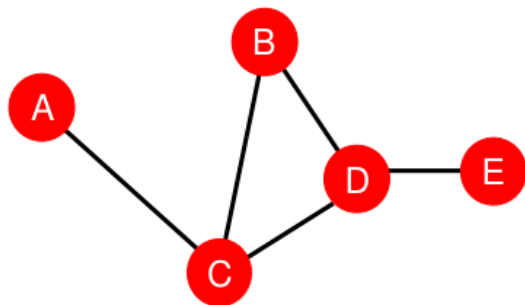
(A-C-D-E, B-D-E, C-D-E)

Closeness centrality:

- A node is important if it has small shortest path lengths to all other nodes.

$$c_v = \frac{1}{\sum_{u \neq v} \text{shortest path length between } u \text{ and } v}$$

- **Example:**



$$c_A = 1/(2 + 1 + 2 + 3) = 1/8$$

(A-C-B, A-C, A-C-D, A-C-D-E)

$$c_D = 1/(2 + 1 + 1 + 1) = 1/5$$

(D-C-A, D-B, D-C, D-E)

Коэффициент кластеризации вершины

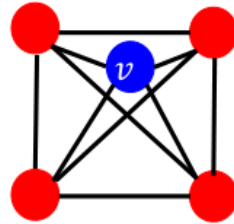
- Measures how connected v 's neighboring nodes are:

$$e_v = \frac{\#(\text{edges among neighboring nodes})}{\binom{k_v}{2}} \in [0,1]$$

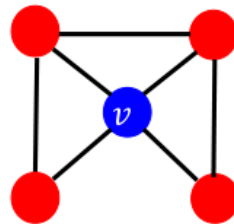
#(node pairs among k_v neighboring nodes)

In our examples below the denominator is 6 (4 choose 2).

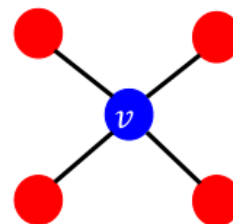
- Examples:**



$$e_v = 1$$



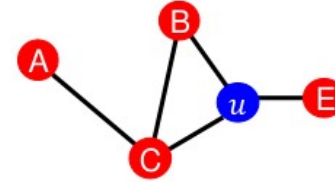
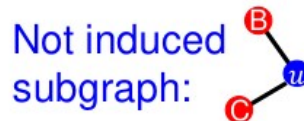
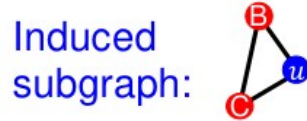
$$e_v = 0.5$$



$$e_v = 0$$

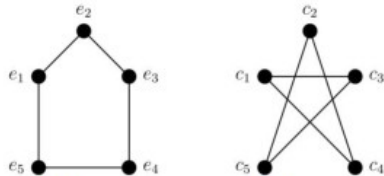
Графлеты ноды

- **Def: Induced subgraph** is another graph, formed from a subset of vertices and *all* of the edges connecting the vertices in that subset.



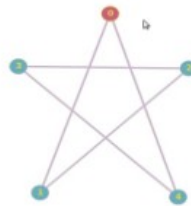
- **Def: Graph Isomorphism**

- Two graphs which contain the same number of nodes connected in the same way are said to be isomorphic.

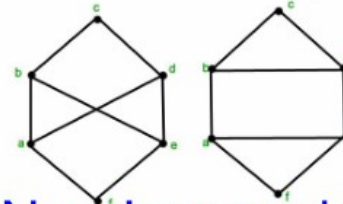


Isomorphic

Node mapping: (e2,c2), (e1, c5), (e3,c4), (e5,c3), (e4,c1)



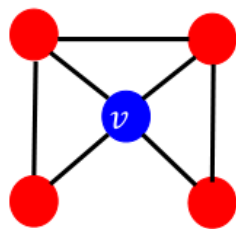
Source: Mathoverflow



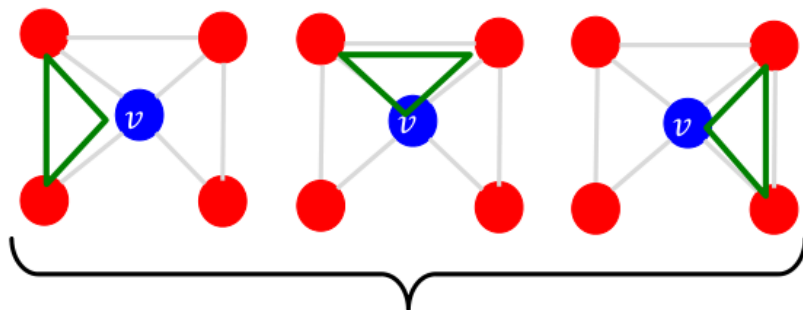
Non-Isomorphic

The right graph has cycles of length 3 but the left graph does not, so the graphs cannot be isomorphic.

- **Observation:** Clustering coefficient counts the #(triangles) in the ego-network



$$e_v = 0.5$$

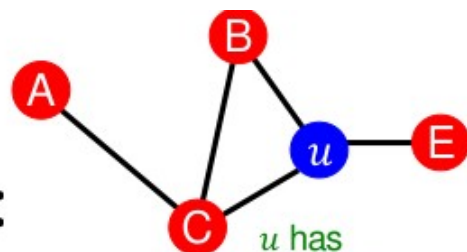


3 triangles (out of 6 node triplets)

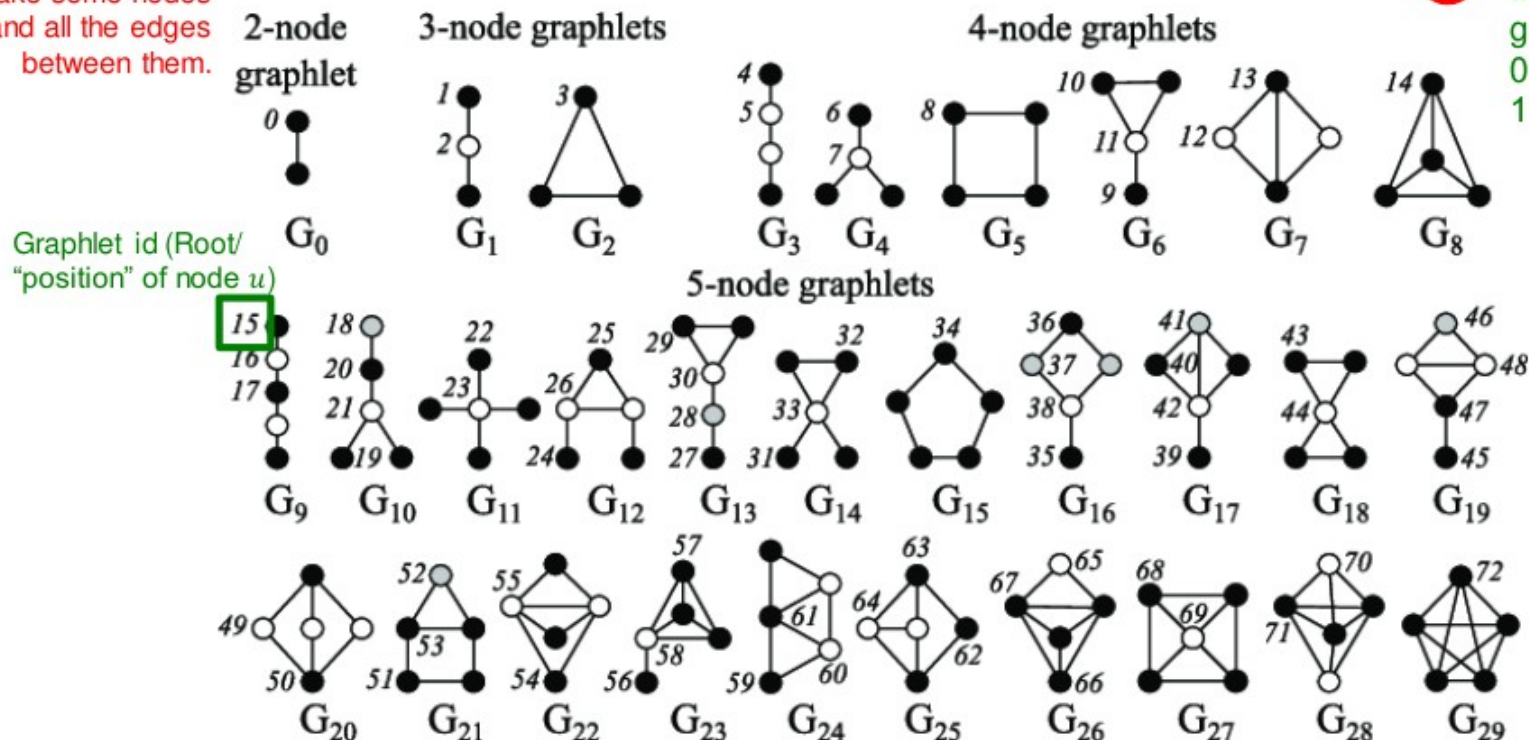
- We can generalize the above by counting #(pre-specified subgraphs, i.e., **graphlets**).

Graphlets: Rooted connected induced non-isomorphic subgraphs:

Take some nodes and all the edges between them.



u has graphlets: 0, 1, 2, 3, 5, 10, 11, ...



There are 73 different graphlets on up to 5 nodes

2-node graphlet embedding
1D



2-3 node graphlet embedding
4D



2-4 node graphlet embedding
15D



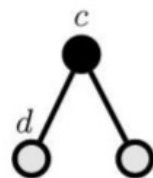
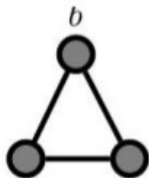
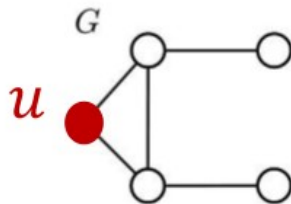
2-5 node graphlet embedding
73D



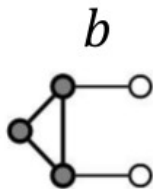
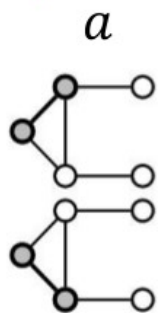
- **Graphlet Degree Vector (GDV):** A count vector of graphlets rooted at a given node.

- **Example:**

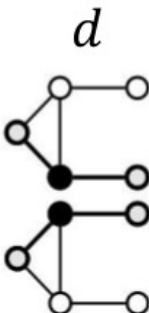
Possible graphlets up to size 3



Graphlet instances of node u :



c



GDV of node u :

a, b, c, d

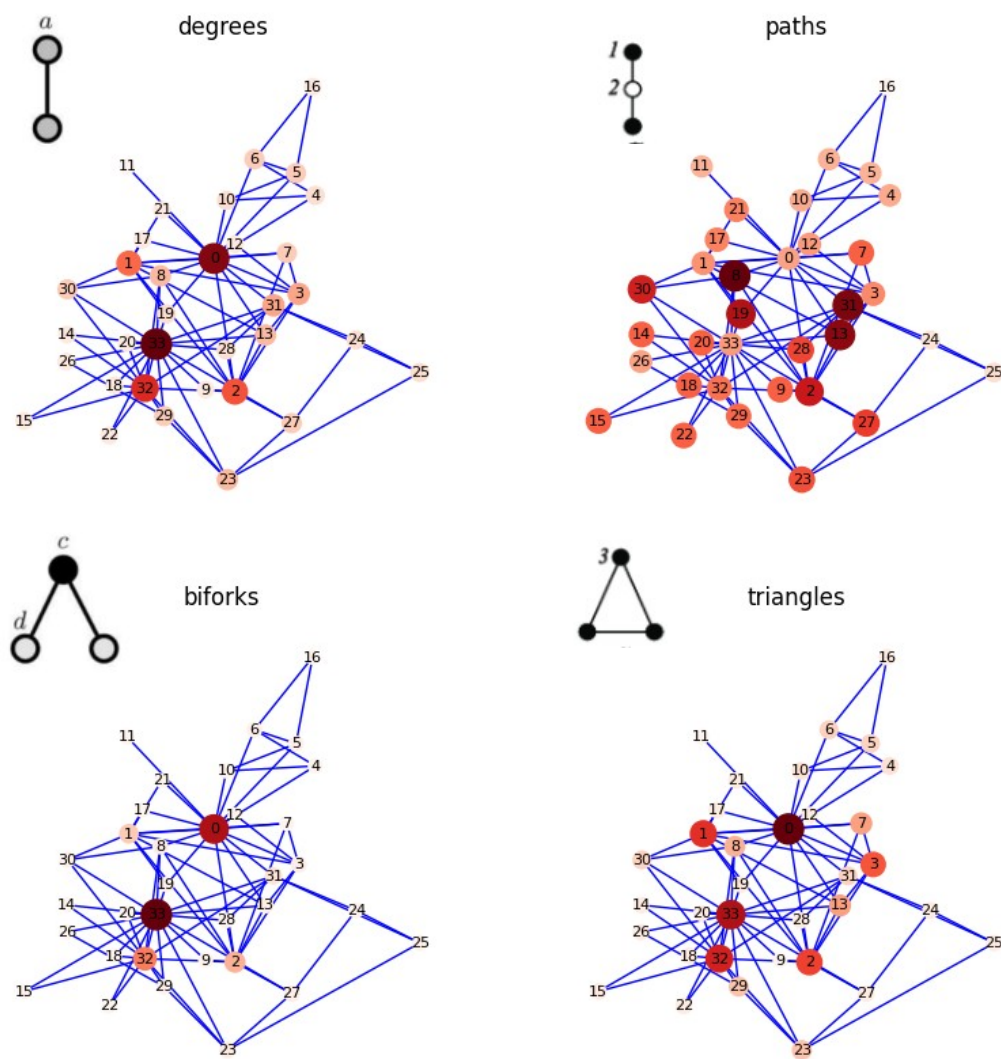
$[2, 1, 0, 2]$

Вектор графлета кодирует локальную топологию (локальную структуру) графа вблизи изучаемой ноды.

GDV[:4]

!pip install pyfgl networkx matplotlib seaborn

```
import networkx as nx
import matplotlib.pyplot as plt
seed = 42
G = nx.karate_club_graph()
import pyfgl.fgl as fg
F = fg.compute(G)
F=np.array(F)
pos = nx.kamada_kawai_layout(G)
degrees = [F[node,1] for node in G.nodes()]
paths = [F[node,2] for node in G.nodes()]
biforks = [F[node,3] for node in G.nodes()]
triangles = [F[node,4] for node in G.nodes()]
labels=['degrees','paths','biforks','triangles']
plt.figure(figsize=(10,10))
for i,gl in enumerate((degrees,paths,biforks,triangles)):
    plt.subplot(2, 2, i+1)
    plt.title(labels[i])
    nx.draw(G, pos,
            node_color=gl,
            cmap=plt.cm.Reds,
            node_size=[50 + (200*v)/np.array(gl).max() for v in gl],
            with_labels=True,
            font_size=8,
            edge_color='blue')
```



Задание: Кластеризовать ноды в их графлет-представлении


```

from sklearn.decomposition import PCA

pca=PCA(n_components=2)

X=pca.fit_transform(F)

from sklearn.mixture import GaussianMixture

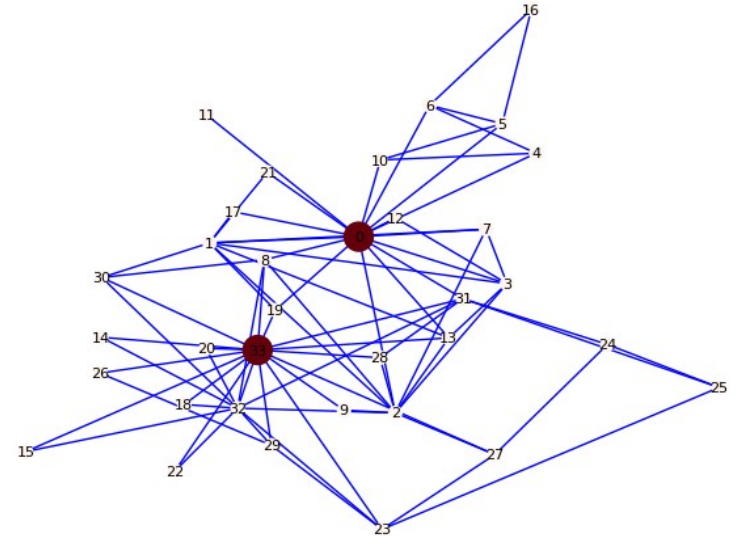
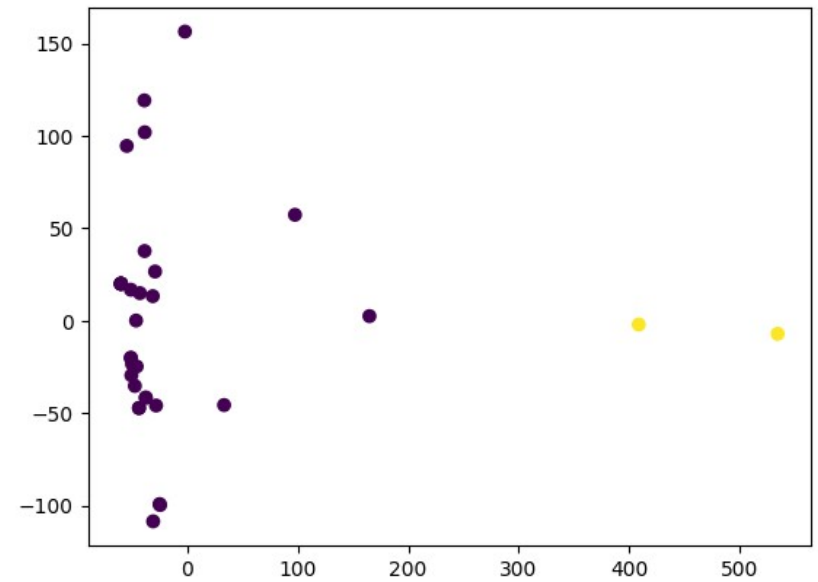
clf=GaussianMixture(n_components=2)

c=clf.fit_predict(X)

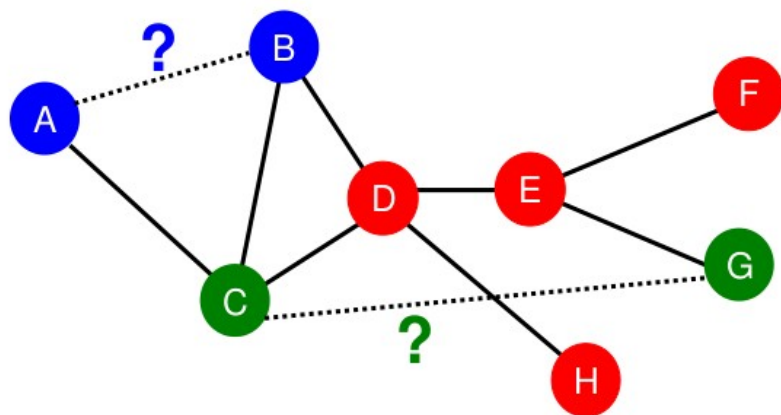
plt.scatter(X[:,0],X[:,1],c=c)

nx.draw(G, pos,
        node_color=c,
        cmap=plt.cm.Reds,
        node_size=[50 + (200*v) for v in list(c)],
        with_labels=True,
        font_size=8,
        edge_color='blue')

```



Прогноз связей/ребер



The task is to predict **new links** based on the existing links.

At test time, node pairs (with no existing links) are ranked, and top K node pairs are predicted.

The key is to design features for a **pair of nodes**.

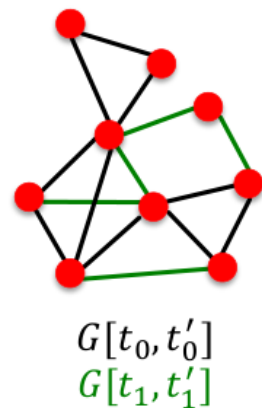
Two formulations of the link prediction task:

■ 1) Links missing at random:

- Remove a random set of links and then aim to predict them

■ 2) Links over time:

- Given $G[t_0, t'_0]$ a graph defined by edges up to time t'_0 , **output a ranked list L** of edges (not in $G[t_0, t'_0]$) that are predicted to appear in time $G[t_1, t'_1]$
- **Evaluation:**
 - $n = |E_{new}|$: # new edges that appear during the test period $[t_1, t'_1]$
 - Take top n elements of L and count correct edges



Классические признаки для ребра

Простейший способ кодировки ребра – объединение эмбединга двух нод, которые это ребро соединяет

graphlet node embedding

bi-graphlet link embedding

4D  Node 1



4D  Node 2

ребро — — — — 

Проблема – эмбединг предполагает, что код независим.

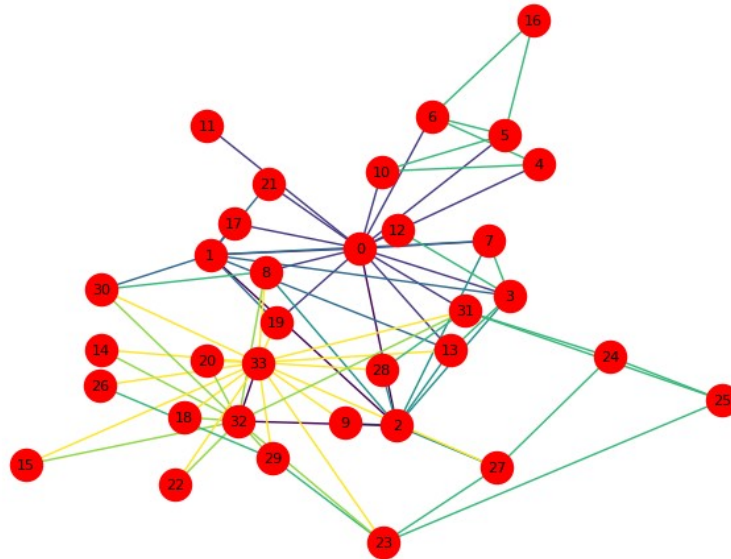
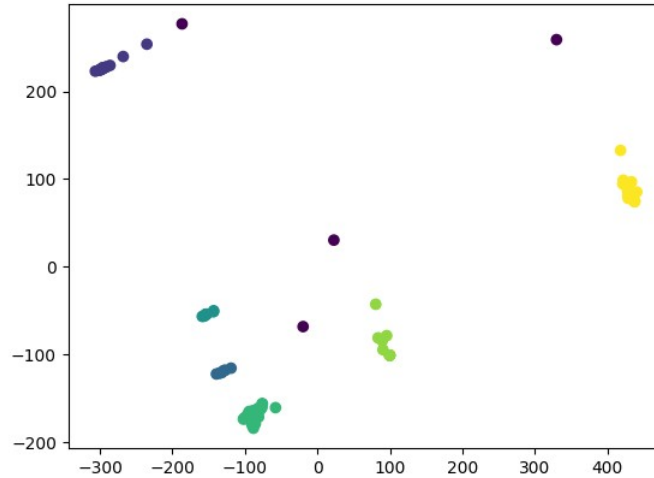
Задача: кластеризовать ребра в их би-графлет представлении. Кластеризовать DBSCAN-ом.

```

import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import pyfgl.fgl as fg
seed = 42
G = nx.karate_club_graph()
pos = nx.kamada_kawai_layout(G)
F = fg.compute(G)
F=np.array(F)
edge_f = []
for u, v in G.edges:
    edge_f.append(np.concatenate([F[u],F[v]]))
edge_f=np.array(edge_f)
print(edge_f.shape)
from sklearn.decomposition import PCA
pca=PCA(n_components=2)
X=pca.fit_transform(edge_f)
from sklearn.cluster import DBSCAN
clf=DBSCAN(eps=50)
c=clf.fit_predict(X)
plt.scatter(X[:,0],X[:,1],c=c)

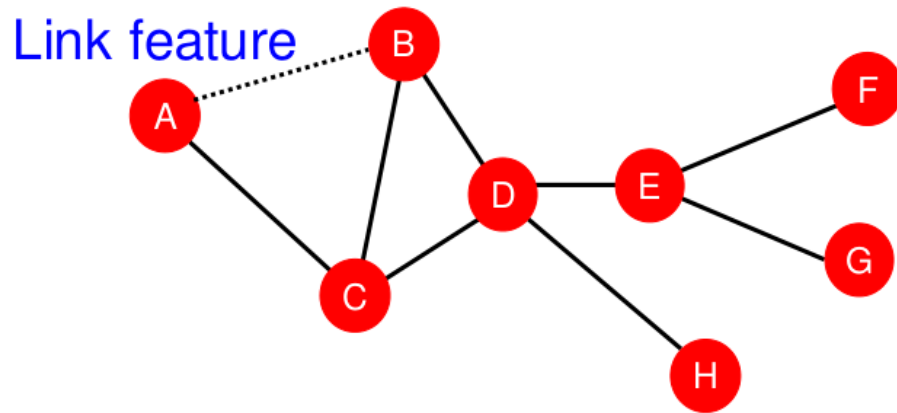
nx.draw(G, pos,
        node_color='red',
        cmap=plt.cm.Reds,
        with_labels=True,
        font_size=8,
        edge_color=c)

```



Классические признаки для ребра

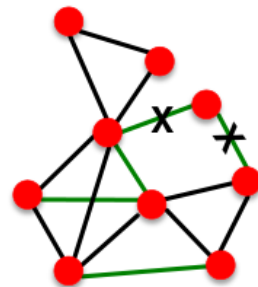
- Distance-based feature
- Local neighborhood overlap
- Global neighborhood overlap



Признаки основанные на расстоянии

Близость нод

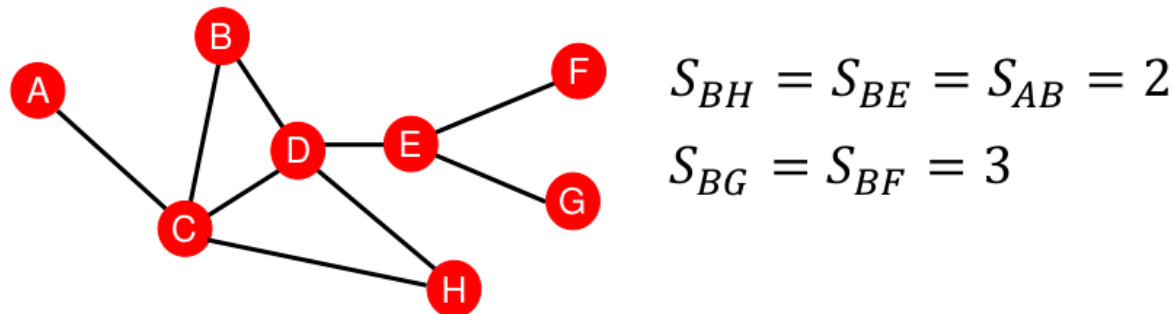
- For each pair of nodes (x,y) compute score $c(x,y)$
 - For example, $c(x,y)$ could be the # of common neighbors of x and y
- Sort pairs (x,y) by the decreasing score $c(x,y)$
- **Predict top n pairs as new links**
- **See which of these links actually appear in $G[t_1, t'_1]$**



Длина кратчайшего пути между узлами

Shortest-path distance between two nodes

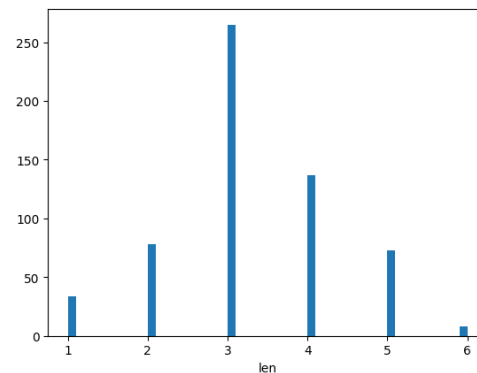
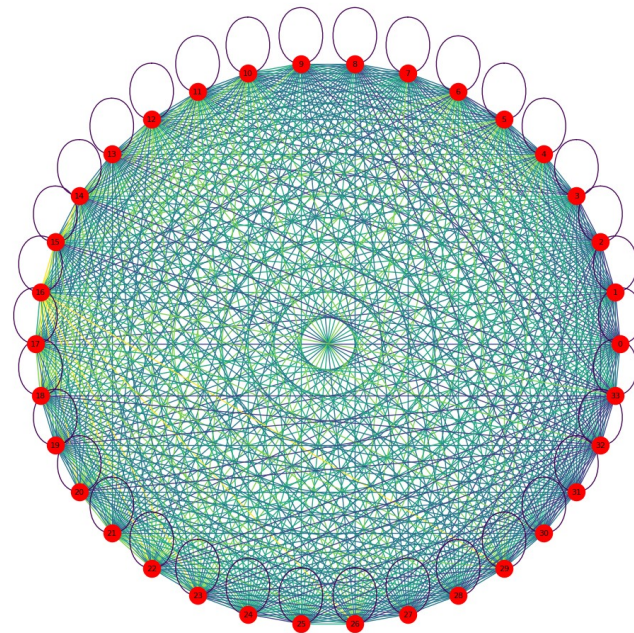
- Example:



- However, this does not capture the degree of neighborhood overlap:
 - Node pair (B, H) has 2 shared neighboring nodes, while pairs (B, E) and (A, B) only have 1 such node.

Длина кратчайшего пути между нодами

```
edge_f = {}
G1=nx.Graph()
for u in G.nodes:
    for v in G.nodes:
        l=len(nx.shortest_path(G,u,v))
        edge_f[tuple((u,v))]=l
        if(100>l>-1):
            G1.add_edge(u,v,weight=l)
pos=nx.circular_layout(G1)
edge_labels = nx.get_edge_attributes(G1, 'weight')
c=[]
for u,v in G1.edges:
    c.append(edge_labels[u,v])
plt.figure(figsize=(10,10))
nx.draw(G1, pos,
        node_color='red',
        cmap=plt.cm.Reds,
        with_labels=True,
        font_size=8,
        edge_color=c)
plt.hist(c,bins=50);
plt.xlabel('len')
```



Перекрытие локальных окрестностей

Captures # neighboring nodes shared between two nodes v_1 and v_2 :

Количество общих соседей

Коэффициент Жаккарда (IoU)

Коэффициент Адамич-Адара (взвешанная обратная степень нод общих соседей)

■ **Common neighbors:** $|N(v_1) \cap N(v_2)|$

■ Example: $|N(A) \cap N(B)| = |\{C\}| = 1$

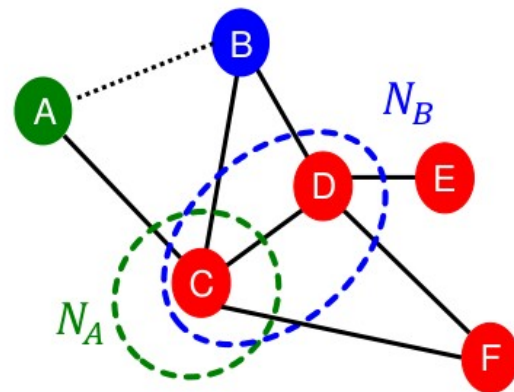
■ **Jaccard's coefficient:** $\frac{|N(v_1) \cap N(v_2)|}{|N(v_1) \cup N(v_2)|}$

■ Example: $\frac{|N(A) \cap N(B)|}{|N(A) \cup N(B)|} = \frac{|\{C\}|}{|\{C, D\}|} = \frac{1}{2}$

■ **Adamic-Adar index:**

$$\sum_{u \in N(v_1) \cap N(v_2)} \frac{1}{\log(k_u)}$$

■ Example: $\frac{1}{\log(k_C)} = \frac{1}{\log 4}$

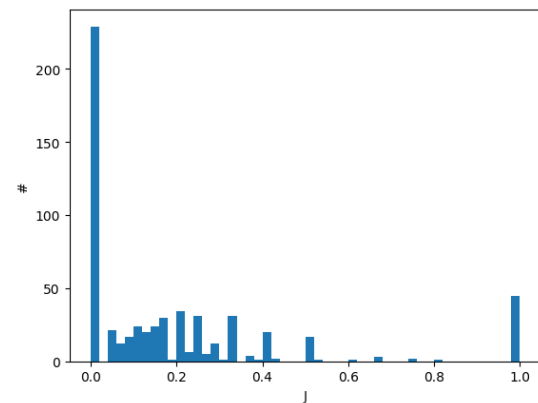
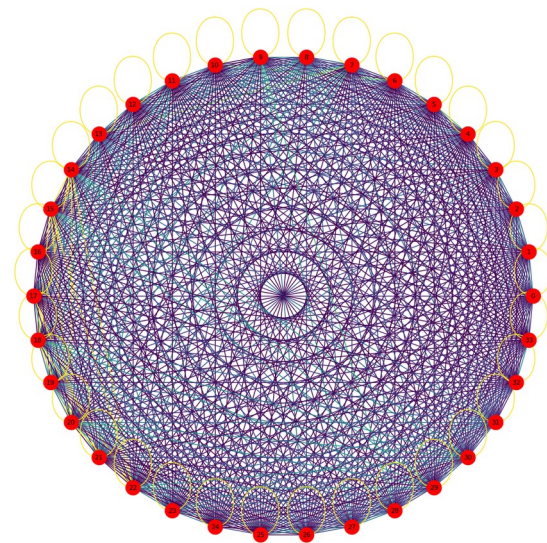


Недостаток: локальность. Плохо работает, когда нет общих соседей (всегда возвращает 0).

Перекрывание локальных окрестностей

```
edge_f = {}
G1=nx.Graph()
for u in G.nodes:
    for v in G.nodes:
        pred=nx.jaccard_coefficient(G,[(u,v)])
        l=list(pred)[0][2]
        edge_f[tuple((u,v))]=l
        G1.add_edge(u,v,weight=l)
pos=nx.circular_layout(G1)
edge_labels = nx.get_edge_attributes(G1, 'weight')
c=[]
for u,v in G1.edges:
    c.append(edge_labels[u,v])
plt.figure(figsize=(10,10))
nx.draw(G1, pos,
        node_color='red',
        cmap=plt.cm.Red,
        with_labels=True,
        font_size=8,
        edge_color=c)
plt.hist(c,bins=50);
plt.xlabel('J')
plt.ylabel('#')
```

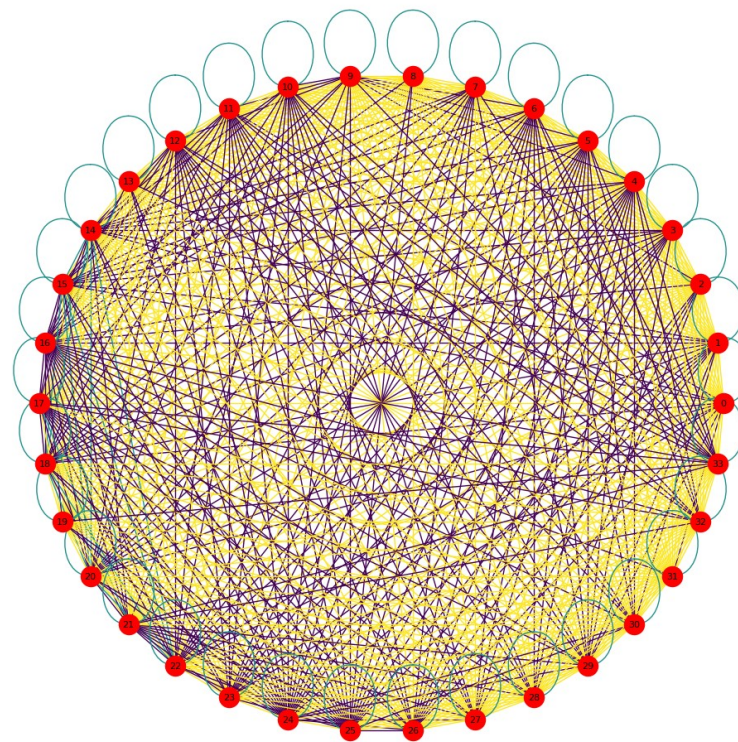
Задание: кластеризовать связи на три кластера –
слабосвязанные, сильносвязанные и среднесвязные.
Использовать GaussianMixture. Отобразить



Перекрывание локальных окрестностей

```
from sklearn.mixture import GaussianMixture
clf=GaussianMixture(n_components=3)
c1=np.array(c)
gmc=clf.fit_predict(c1[:,np.newaxis])
plt.figure(figsize=(10,10))
nx.draw(G1, pos,
        node_color='red',
        cmap=plt.cm.Red,
        with_labels=True,
        font_size=8,
        edge_color=gmc)
```

Задание: выделить сильносвязные линки.
Отобразить

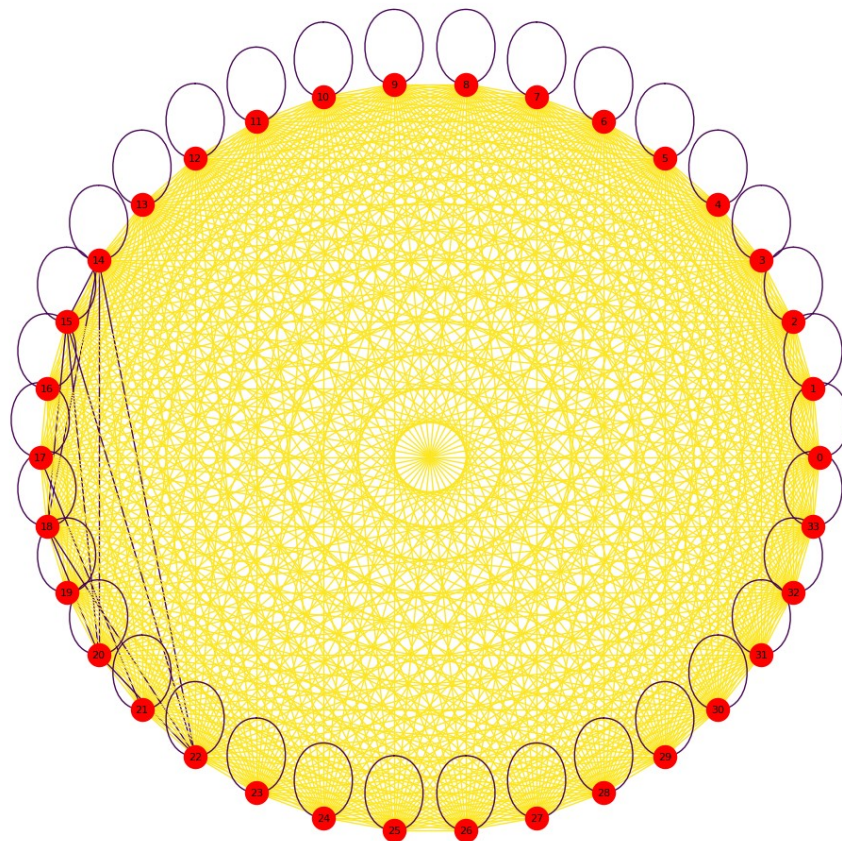


Зеленые – сильносвязные
Желтые – несвязные
Фиолетовые - среднесвязные

Перекрывтие локальных окрестностей

```
plt.figure(figsize=(10,10))
gmc2=gmc.copy()
gmc2[gmc2!=1]=0
gmc2=np.abs(gmc2-1)
nx.draw(G1, pos,
        node_color='red',
        cmap=plt.cm.Red,
        with_labels=True,
        font_size=8,
        edge_color=gmc2)
```

Задача: выделить сильносвязные
линки используя индекс Адамича-
Адара. Нарисовать граф, состоящий
лишь из сильносвязных ребер,
ребра ноды на себя не учитывать.
сравнить с оригинальным графом.



Темные – сильносвязные линки

Перекрывание глобальных окрестностей

Индекс Каца (Katz) – число путей между двумя нодами

Как считать? Через матрицу смежности.

Количество путей длины 1 между двумя нодами $u, v = A_{u,v}$

Количество путей длины 2 между двумя нодами – сумма путей между u, x и x, v : $\sum_x A_{u,x} A_{x,v} = A_{uv}^2$

...

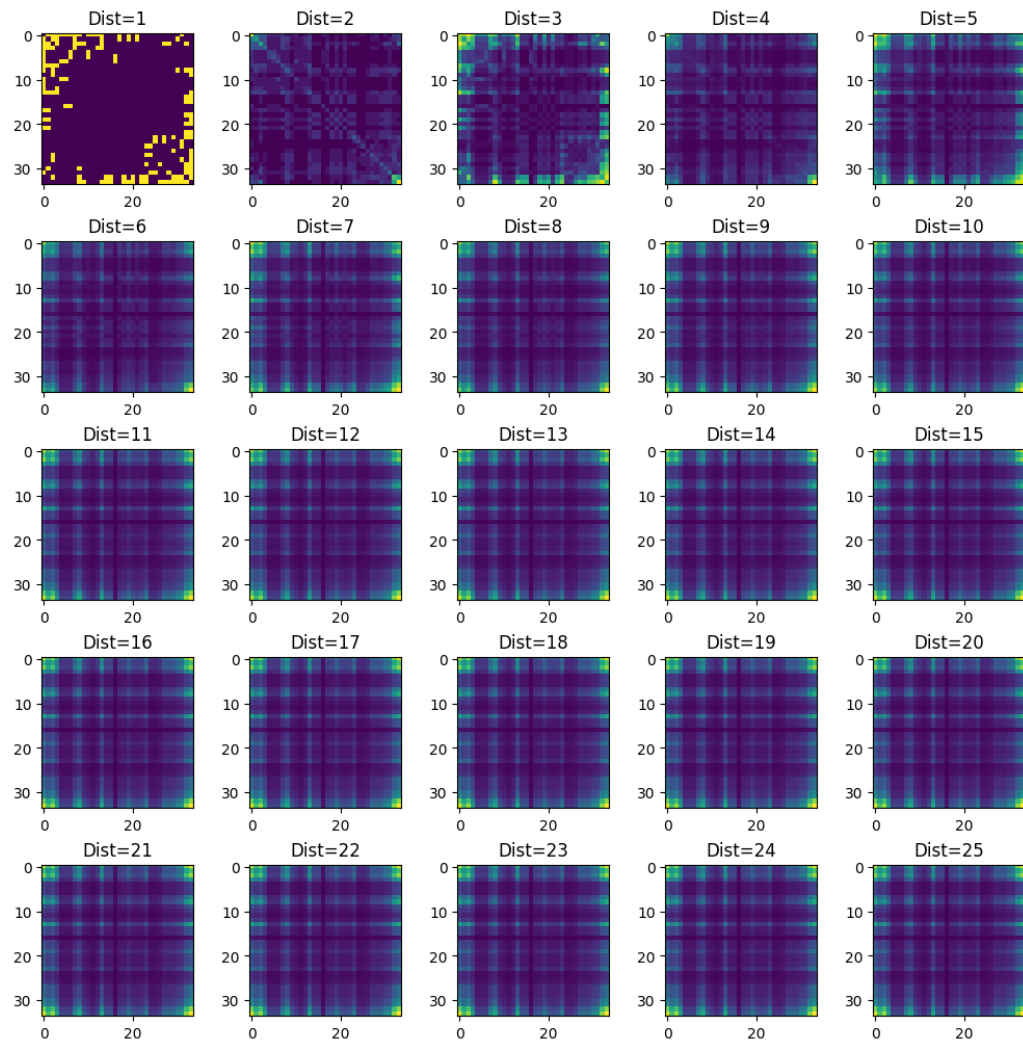
Количество путей длины K между двумя нодами – A_{uv}^K

Тогда полное количество путей между двумя нодами:

$$P_{uv} = \sum_k (A_{uv}^k)$$

Перекрывание глобальных окрестностей

```
AG=nx.adjacency_matrix(G).toarray()
AG[AG>0]=1
AGI=np.diag(np.ones(AG.shape[0]))
plt.figure(figsize=(10,10),constrained_layout=True)
for i in range(25):
    AGI=AGI@AG
    plt.subplot(5,5,i+1)
    plt.imshow(AGI)
    plt.title('Dist='+str(i+1))
```



Перекрывание глобальных окрестностей

- **Katz index** between v_1 and v_2 is calculated as

Sum over *all walk lengths*

$$S_{v_1 v_2} = \sum_{l=1}^{\infty} \boxed{\beta^l} \boxed{A_{v_1 v_2}^l} \begin{array}{l} \text{\#walks of length } l \\ \text{between } v_1 \text{ and } v_2 \end{array}$$

$0 < \beta < 1$: discount factor

- Katz index matrix is computed in closed-form:

$$S = \sum_{i=1}^{\infty} \beta^i A^i = \underbrace{(I - \beta A)^{-1}}_{= \sum_{i=0}^{\infty} \beta^i A^i} - I,$$

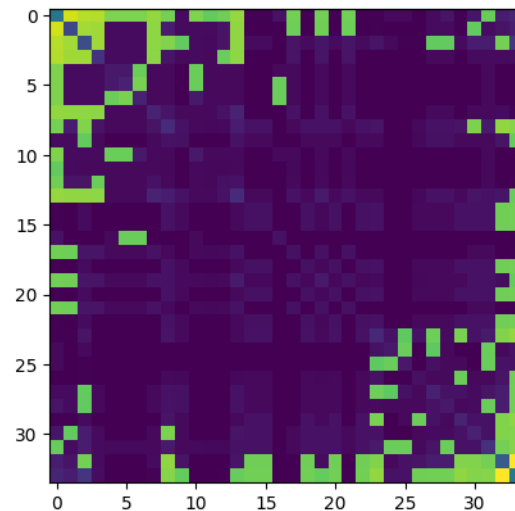
by geometric series of matrices

Это – известная сумма бесконечной геометрической прогрессии.
Таким образом, индекс Каца сводится к вычислению обратной матрицы.

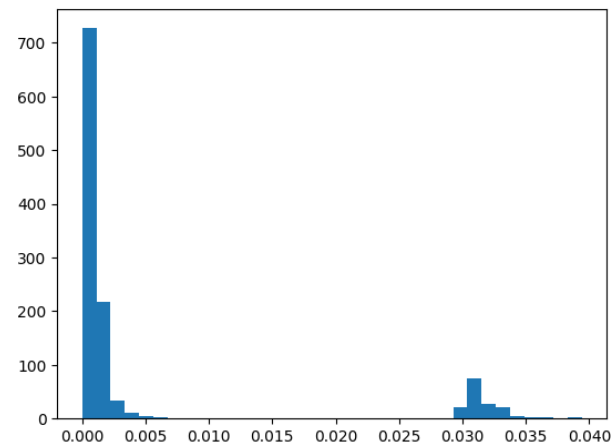
Чтобы решение сходилось, необходимо: $\beta < 1/\lambda_{max}$

Перекрывание глобальных окрестностей

```
beta=1/AG.shape[0]  
I=np.diag(np.ones(AG.shape[0]))  
result=np.linalg.inv(I - (beta * AG))-I  
plt.imshow(result)  
  
plt.hist(result.reshape(-1,1),bins=35);
```



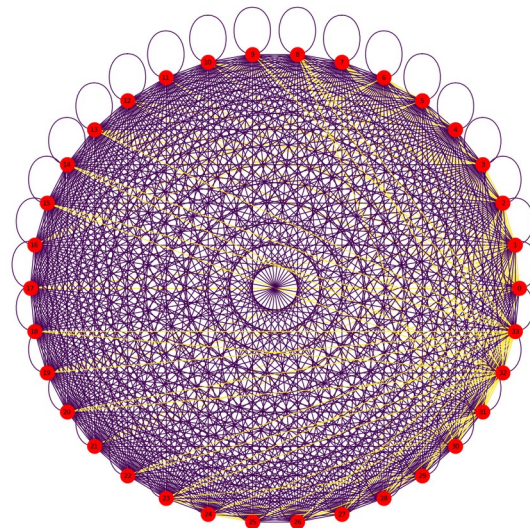
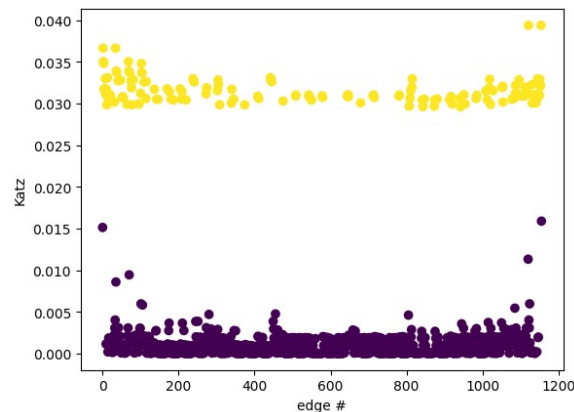
Задание: разделить на близкие (K мал) и далекие (K велик) связи. Отобразить



Перекрывание глобальных окрестностей

```
t1=np.zeros(result.shape[0]*result.shape[1])
f=result.reshape(-1,1)[:,:0]
t1[f>0.025]=1
t1[f<=0.025]=0
plt.scatter(range(result.shape[0]*result.shape[0]),
result.reshape(-1,1),c=t1)
plt.xlabel('edge #')
plt.ylabel('Katz')
t=t1.reshape(result.shape[0],result.shape[1])

edge_f = {}
G1=nx.Graph()
for u in G.nodes:
    for v in G.nodes:
        l=t[u,v]
        edge_f[tuple((u,v))]=l
        G1.add_edge(u,v,weight=l)
pos=nx.circular_layout(G1)
edge_labels = nx.get_edge_attributes(G1, 'weight')
c=[]
for u,v in G1.edges:
    c.append(edge_labels[u,v])
plt.figure(figsize=(10,10))
nx.draw(G1, pos, node_color='red', cmap=plt.cm.Reds,
        with_labels=True, font_size=8,edge_color=c)
```



Классические признаки для графа

Ядерные методы

- Kernel $K(G, G') \in \mathbb{R}$ measures similarity b/w data
- Kernel matrix $\mathbf{K} = (K(G, G'))_{G, G'}$, must always be positive semidefinite (i.e., has positive eigenvalues)
- There exists a feature representation $\phi(\cdot)$ such that $K(G, G') = \phi(G)^T \phi(G')$
- Once the kernel is defined, off-the-shelf ML model, such as **kernel SVM**, can be used to make predictions.
- **Graphlet Kernel [1]**
- **Weisfeiler-Lehman Kernel [2]**
- Other kernels are also proposed in the literature (beyond the scope of this lecture)
 - Random-walk kernel
 - Shortest-path graph kernel
 - And many more...

Таким образом, нахождение ядра автоматически будет соответствовать нахождению графового представления такого, что скалярное произведение представлений двух графов будет давать ядро.

Идея графовых ядер

- **Goal:** Design graph feature vector $\phi(G)$
- **Key idea:** Bag-of-Words (BoW) for a graph
 - **Recall:** BoW simply uses the word counts as features for documents (no ordering considered).
 - Naïve extension to a graph: **Regard nodes as words.**
 - Since both graphs have **4 red nodes**, we get the same feature vector for two different graphs...

$$\phi(\text{Graph 1}) = \phi(\text{Graph 2})$$

Если просто считать ноды и их внутренние характеристики, то даже непохожие графы будут иметь одинаковые признаки

What if we use Bag of **node degrees**?

Deg1: ● Deg2: ● Deg3: ●

$$\phi(\text{Graph 1}) = \text{count}(\text{Graph 1}) = [1, 2, 1]$$

$$\phi(\text{Graph 2}) = \text{count}(\text{Graph 2}) = [0, 2, 2]$$

Obtains different features for different graphs!

- Both Graphlet Kernel and Weisfeiler-Lehman (WL) Kernel use **Bag-of-*** representation of graph, where ***** is more sophisticated than node degrees!

Если учитывать хотя-бы степени вершин, то можно различить графы!

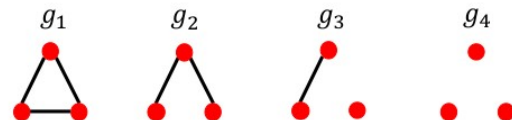
Графлет-ядро

Key idea: Count the number of different graphlets in a graph.

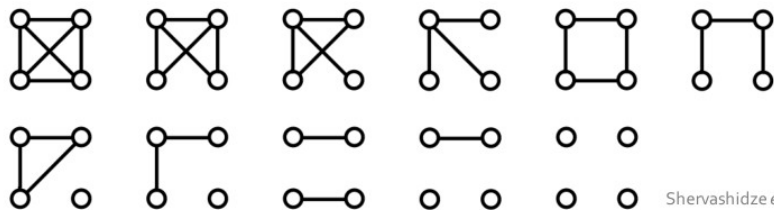
- **Note:** Definition of graphlets here is slightly different from node-level features.
- The two differences are:
 - Nodes in graphlets here do **not need to be connected** (allows for isolated nodes)
 - The graphlets here are not rooted.
 - Examples in the next slide illustrate this.

Let $\mathcal{G}_k = (g_1, g_2, \dots, g_{n_k})$ be a list of graphlets of size k .

- For $k = 3$, there are 4 graphlets.



- For $k = 4$, there are 11 graphlets.

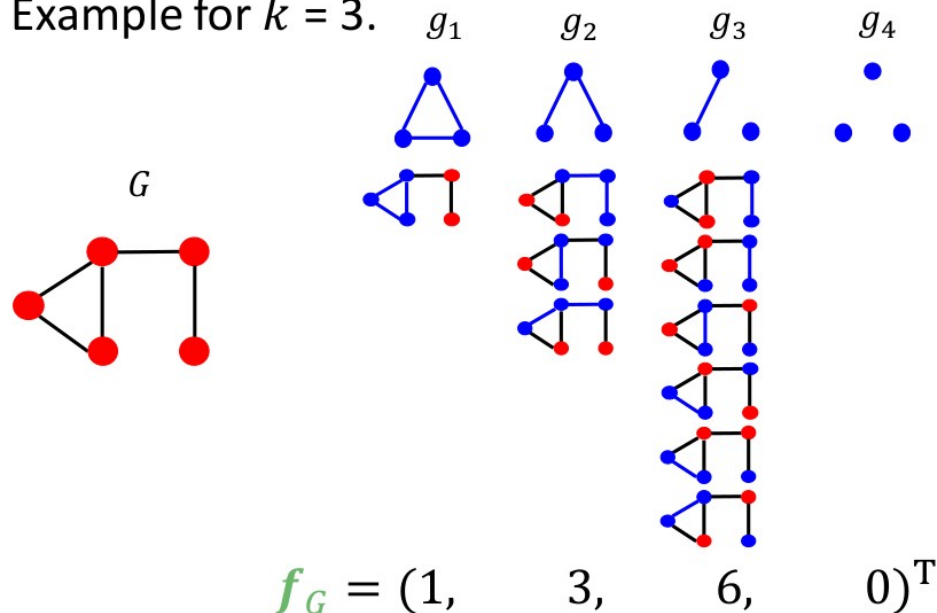


Графлет-ядро

Given graph G , and a graphlet list $\mathcal{G}_k = (g_1, g_2, \dots, g_{n_k})$, define the graphlet count vector $\mathbf{f}_G \in \mathbb{R}^{n_k}$ as

$$(\mathbf{f}_G)_i = \#(g_i \subseteq G) \text{ for } i = 1, 2, \dots, n_k.$$

Example for $k = 3$.



Given two graphs, G and G' , graphlet kernel is computed as

$$K(G, G') = \mathbf{f}_G^T \mathbf{f}_{G'}$$

Problem: if G and G' have different sizes, that will greatly skew the value.

Solution: normalize each feature vector

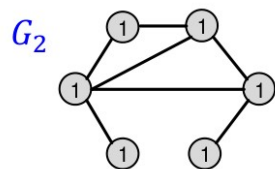
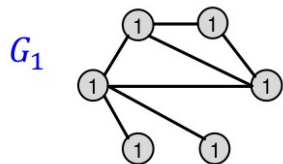
$$\mathbf{h}_G = \frac{\mathbf{f}_G}{\text{Sum}(\mathbf{f}_G)} \quad K(G, G') = \mathbf{h}_G^T \mathbf{h}_{G'}$$

Limitations: Counting graphlets is **expensive!**

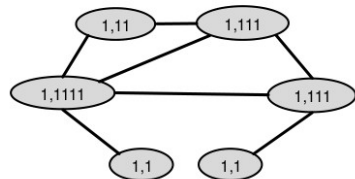
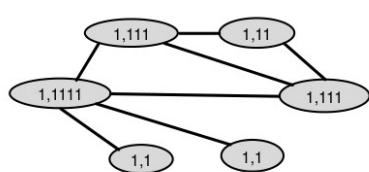
- Counting size- k graphlets for a graph with size n by enumeration takes n^k .
- This is unavoidable in the worst-case since **subgraph isomorphism test** (judging whether a graph is a subgraph of another graph) is **NP-hard**.
- If a graph's node degree is bounded by d , an $O(nd^{k-1})$ algorithm exists to count all the graphlets of size k .

Ядро Вайсфайлера-Лемана (Weisfeiler-Lehman) или уточнение цвета

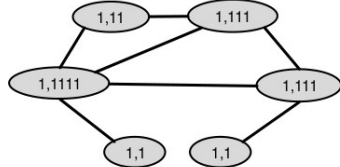
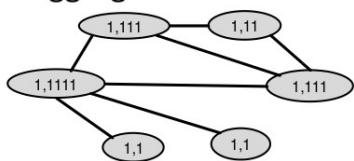
- Assign initial colors



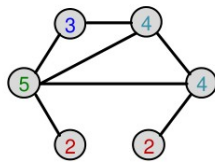
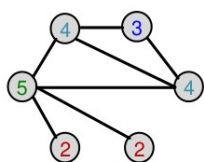
- Aggregate neighboring colors



- Aggregated colors



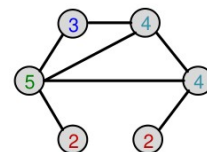
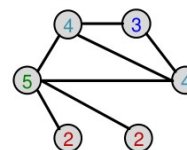
- Hash aggregated colors



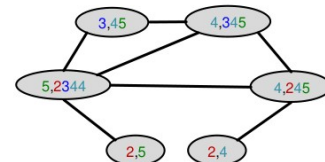
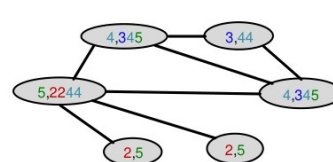
Hash table

1,1	-->	2
1,11	-->	3
1,111	-->	4
1,1111	-->	5

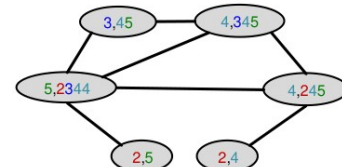
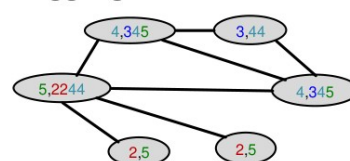
- Aggregated colors



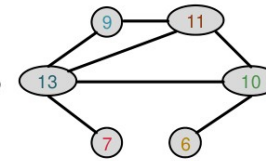
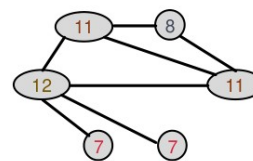
- Hash aggregated colors



- Aggregated colors



- Hash aggregated colors



Hash table

2,4	-->	6
2,5	-->	7
3,44	-->	8
3,45	-->	9
4,245	-->	10
4,345	-->	11
5,2244	-->	12
5,2344	-->	13

Ядро Вайсфайлера-Лемана (Weisfeiler-Lehman) или уточнение цвета

After color refinement, WL kernel counts number of nodes with a given color.

$$\phi(\text{Graph 1}) = \begin{matrix} \text{Colors} \\ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 \\ \text{Counts} \\ [6, 2, 1, 2, 1, 0, 2, 1, 0, 0, 0, 2, 1] \end{matrix}$$

$$\phi(\text{Graph 2}) = \begin{matrix} 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13 \\ [6, 2, 1, 2, 1, 1, 1, 0, 1, 1, 1, 0, 1] \end{matrix}$$

$$\begin{aligned} K(\text{Graph 1}, \text{Graph 2}) &= \phi(\text{Graph 1})^T \phi(\text{Graph 2}) \\ &= 49 \end{aligned}$$

WL kernel is **computationally efficient**

- The time complexity for color refinement at each step is linear in $\#(\text{edges})$, since it involves aggregating neighboring colors.

When computing a kernel value, only colors appeared in the two graphs need to be tracked.

- Thus, $\#(\text{colors})$ is at most the total number of nodes.

Counting colors takes linear-time w.r.t. $\#(\text{nodes})$.

In total, time complexity is **linear in $\#(\text{edges})$** .

Ядро Вайсфайлера-Лемана (Weisfeiler-Lehman) или уточнение цвета

```
import networkx as nx
import numpy as np
```

```
def get_hashes(list_labels,hashes=[]):
    idt={}
    for labels in list_labels:
        for k in list(labels.keys()):
            if not (labels[k] in idt):
                idt[labels[k]]=1
    hashes=[k for k in idt.keys()]
    return hashes
```

```
def weisfeiler_lehman(G, max_iter=50):
    labels = {n: hash(tuple([0])) for n in G.nodes()}
    for itt in range(max_iter):
        new_labels = {
            n: hash (tuple([labels[n]]+sorted(labels[v] for v in G[n])))
            for n in G}
        hashes=get_hashes([new_labels])
        idt={v:k for k,v in enumerate(hashes)}
        new_labels_emb = {n: idt[new_labels[n]] for n in G}
        new_labels=new_labels_emb
        if new_labels == labels: break
        labels = new_labels
    return labels
```

```
def emb(labels,hashes):
    i=len(hashes)
    idt={v:k for k,v in enumerate(hashes)}
    wl_emb=np.zeros(i).astype(int)
    for k in list(labels.keys()):
        if labels[k] in hashes:
            wl_emb[idt[labels[k]]]+=1
    return wl_emb
```

```
G1 = nx.karate_club_graph() # C4 graph
labels1 = weisfeiler_lehman(G1, max_iter=20)
G2 = nx.cycle_graph(32)
labels2 = weisfeiler_lehman(G2, max_iter=100)
G3 = nx.cycle_graph(35)
labels3 = weisfeiler_lehman(G3, max_iter=100)
G4 = nx.cycle_graph(5)
labels4 = weisfeiler_lehman(G4, max_iter=100)
hashes=get_hashes([labels1,labels2,labels3,labels4])
print(emb(labels1,hashes))
print(emb(labels2,hashes))
print(emb(labels3,hashes))
print(emb(labels4,hashes))
```

```
[1 1 1 1 2 2 1 1 1 1 1 1 5 1 2 1 1 1 1 1 1 1 1 1 1 1]
```

```
[32 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

```
[35 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

```
[5 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
```

- **Graphlet Kernel**

- Graph is represented as **Bag-of-graphlets**
- **Computationally expensive**

- **Weisfeiler-Lehman Kernel**

- Apply K -step color refinement algorithm to enrich node colors
 - Different colors capture different K -hop neighborhood structures
- Graph is represented as **Bag-of-colors**
- **Computationally efficient**
- Closely related to Graph Neural Networks (as we will see!)